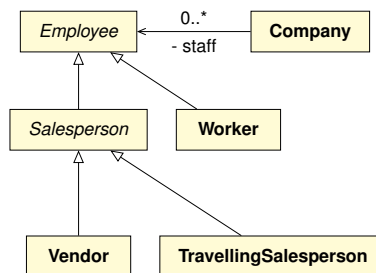


Exercice 1 : Calcul de salaires

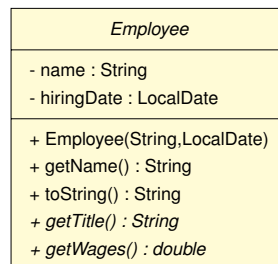
Une entreprise regroupe différents types de salariés qui ne sont pas forcément soumis aux mêmes règles de calcul du salaire. Par exemple :

- les *vendeurs* (*Vendor*) reçoivent une commission de 20% du chiffre d'affaires qu'ils ramènent en plus d'un bonus de 400 €;
- les *représentants* (*Salesperson*) reçoivent, eux aussi, une commission de 20% du chiffre d'affaires qu'ils gèrent, mais bénéficient d'un bonus de 800 € en raison de leurs nombreux déplacements;
- les *ouvriers* (*Worker*) sont affectés à la production et sont payés au nombre d'unités qu'ils ont pu produire à raison de 5 €/unité.

Afin de gérer le personnel (c'est-à-dire, l'ensemble des employés), il faut pouvoir ajouter et supprimer des employés, déterminer le salaire de chacun ou le salaire moyen dans l'entreprise. . . Votre code se basera sur la hiérarchie de classes suivante qui comprend initialement 6 classes dont 2 abstraites.

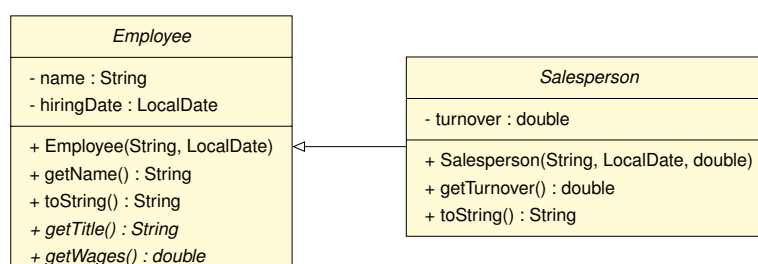


Q1. Écrivez la classe abstraite `Employee` regroupant les caractéristiques et les traitements communs à tous les employés quelle que soit leur fonction.

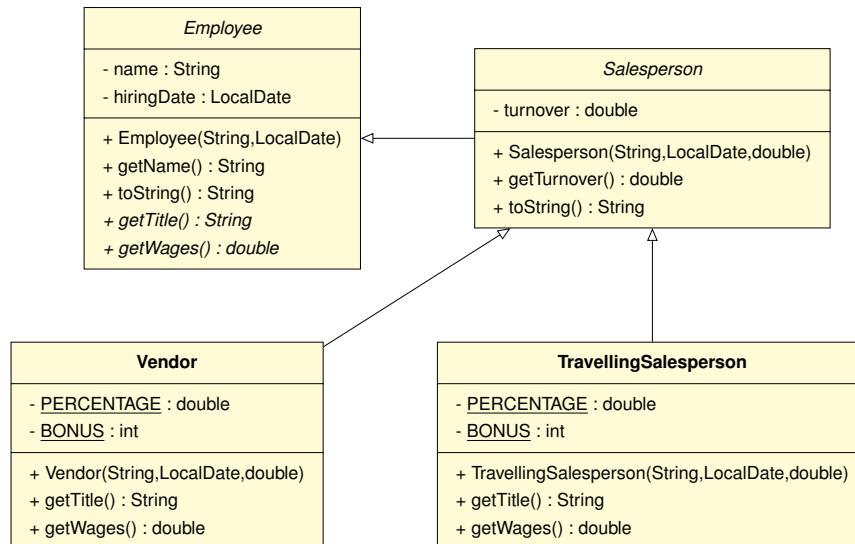


Certaines méthodes comme `getTitle()` (qui, lorsque définie, renvoie une chaîne de caractères contenant le type d'employé) n'ont pour but que de factoriser du code en étant redéfinies dans les classes dérivées et invoquées dans la méthode `toString()`. La méthode `toString()` invoque `getTitle()` et `getName()` : certaines parties sont génériques comme le nom alors que d'autres, comme le type d'employé, sont spécifiques à chaque classe.

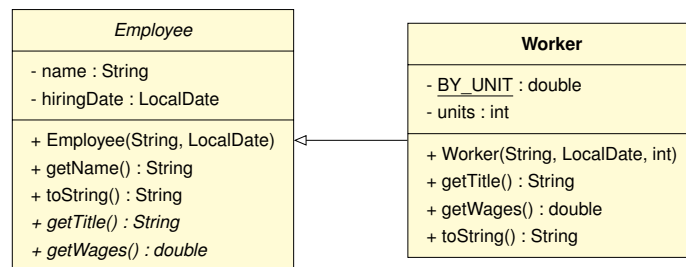
Q2. Les vendeurs et les représentants ayant des points en commun, une classe abstraite `Salesperson` regroupant ces points communs peut être créée. Écrivez cette classe `Salesperson`, dérivant de `Employee`, qui est caractérisée en plus par un chiffre d'affaires (`turnover`).



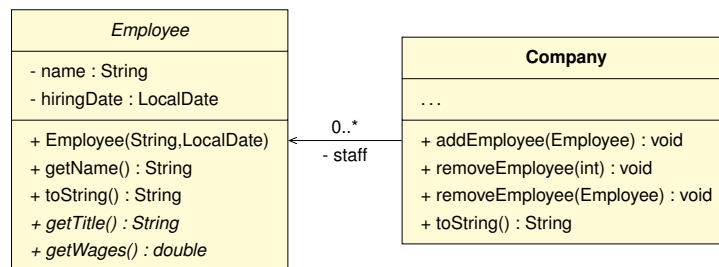
Q3. Écrivez les classes `Vendor` et `TravellingSalesperson` qui dérivent de `Salesperson` en sachant qu'un vendeur touche 20% de commission sur le chiffre d'affaires ramené en plus d'un bonus de 400 € alors que le bonus d'un représentant est de 800 €.



Q4. Ajoutez maintenant la classe `Worker` qui hérite de `Employee` en sachant que les ouvriers sont payés à la pièce produite (5 € l'unité produite).



Q5. Écrivez une classe `Company` qui regroupe un certain nombre d'employés. Ajoutez les méthodes nécessaires pour ajouter ou supprimer des employés.



Q6. On souhaite pouvoir compter le nombre total d'employés, le nombre de personnels de vente (`SalesPerson`) et le nombre d'ouvriers (`Worker`). Ajoutez à la classe `Company` les méthodes de signature :

```

int getNbEmployee()
int getNbSalesPerson()
int getNbWorker()
  
```

Q7. On souhaite pouvoir licencier des employés. Le critère de licenciement utilisé par défaut est toujours la date d'embauche : on licencie les derniers embauchés. Ajoutez ensuite à la classe `Company` une méthode `firing()` qui licencie tous les employés embauchés après la date spécifiée.

```

void firing(LocalDate fatefulDate)
  
```

Q8. Au lieu de se baser sur la date d'embauche, on souhaite pouvoir licencier les employés les moins efficaces.

Q8.1. Ajoutez un attribut statique `objective` de type `double` pour les commerciaux (`Salesperson`) et qui vaut 10000.0. Cet attribut doit être de type `int` pour les ouvriers (`Worker` et valoir 1000).

Q8.2. Ajoutez là où c'est nécessaire une méthode permettant de savoir si un employé donné a atteint son objectif.

```
boolean objectiveFulfilled()
```

Q8.3. Surchargez la méthode `firing` de la classe `Company` pour que, invoquée sans paramètre, elle licencie tous les employés insatisfaits.

```
void firing()
```

Q9. Récupérer la classe de tests `CompanyTest` et valider le fonctionnement de vos méthodes de licenciement.

Q10. Réalisez un `commit` associé au message : "tp00-06::exo-salaire".

Exercice 2 : Revente de véhicules

On souhaite disposer d'un outil permettant de calculer le prix de revente d'une voiture en fonction de son utilisation. On distingue différentes catégories de véhicules et on s'intéresse notamment aux véhicules de collection qui bénéficient d'un mode de calcul spécifique pour leur prix de revente.

Un prix de revente est basé sur le prix d'achat auquel s'applique un taux de décote. Ce taux dépend du kilométrage du véhicule ainsi que de son âge. Par exemple, un véhicule peut perdre 5% de sa valeur (son prix d'achat) en raison de son kilométrage et 7% de plus en raison de son âge important.

Q1. Une voiture est caractérisée par des constantes que sont la marque, la date de première mise en circulation et le prix d'achat, ainsi que par le kilométrage. Écrivez la structure de la classe `Car`, en précisant la nature et la visibilité de chacun des attributs.

Car
?? BRAND : String
?? ON_ROAD : LocalDate
?? PURCHASE_PRICE : double
?? mileage : int
...
...

Q2. Munissez la classe `Car` d'un constructeur complètement spécifié.

```
Car(String brand, LocalDate onRoad, double purchasePrice, int mileage)
```

Q3. Écrivez les accesseurs et les mutateurs possibles pour les attributs `brand`, `onRoad`, `purchasePrice` et `mileage`.

Q4. Deux facteurs de décote sont pris en compte : la décote due au kilométrage (fixée à 0.002) et la décote due à l'âge (fixée à 0.001). Ils serviront à déterminer le prix de vente d'une voiture. Ajoutez un accesseur pour chacune de ses valeurs.

```
double getTemporalDropRate()  
double getMileageDropRate()
```

Q5. Écrivez une méthode `computeDropRate()` retournant le taux cumulé de décote que subit un véhicule en fonction de son âge et du nombre de kilomètres inscrit au compteur. Il faut savoir qu'un véhicule perd `temporalDropRate%` de sa valeur initiale par mois et `mileageDropRate%` de sa valeur tous les 1000 kilomètres parcourus.

```
double computeDropRate()
```

Q6. Ajoutez la méthode `getSalePrice` qui retourne le prix de vente du véhicule.

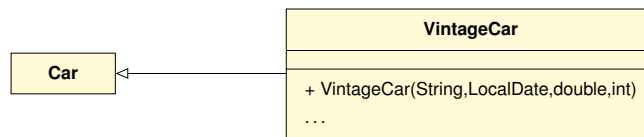
```
double getSalePrice()
```

Q7. Adjoignez une méthode `toString()` retournant une voiture sous la forme textuelle suivante :

```
Car [<brand>, <onRoad>, <mileage>km, <salePrice>euros]
```

On souhaite désormais prendre en compte le cas particulier des voitures de collection. Ces véhicules n'ont pas le même mode de calcul du prix de vente : le taux de décote cumulé ne prend plus en compte les années, mais est uniquement basé sur les kilomètres parcourus.

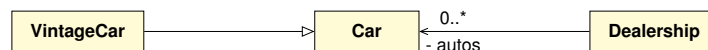
Q8. Écrivez une classe `VintageCar` qui hérite de `Car`, munie d'un constructeur complètement spécifié. Que faut-il redéfinir (un attribut ? une méthode ?) afin de calculer le prix de vente correct pour une voiture de collection.



Q9. Écrivez une classe `UseCar` munie d'un scénario créant deux voitures de mêmes caractéristiques, l'une étant une voiture standard alors que l'autre est un véhicule de collection. Affichez-les. Qu'obtenez-vous ? Pourquoi ?

Comme nous venons nous-même de le faire, nous avons créé une voiture de collection et une voiture standard ayant les mêmes caractéristiques. Ce n'est normalement pas possible, un véhicule de collection doit avoir au minimum 40 ans. Un certain nombre d'utilisateurs ne connaissent pas cette règle ou cherchent délibérément à contourner la règle de classification afin de maximiser le prix de vente. Intéressons-nous à la concession cherchant à éviter ce phénomène.

Q10. Écrivez une classe `Dealership` qui contient des voitures. Munissez cette classe d'une méthode nécessaire à l'ajout d'un véhicule donné à ceux qu'elle gère.



```
void addCar(Car v)
```

Q11. On souhaite retirer à l'utilisateur le droit de créer des instances de `Car` et de `VintageCar` pour éviter toute manipulation (intentionnelle ou non). En effet, un utilisateur pourrait choisir une instance de `VintageCar` au lieu de `Car` pour minimiser la décote qui serait appliquée. Écrivez au sein de la classe `Dealership` les méthodes de classe `createCar()` qui retournent une instance de l'objet adéquat en fonction des paramètres fournis. Si la voiture a plus de 40 ans, une instance de `VintageCar` doit être retournée sinon une instance de `Car`. On veut conserver la même souplesse au niveau des paramètres attendus que celle qu'avaient les constructeurs (les valeurs par défaut).

Q12. Réalisez un `commit` associé au message : `"tp00-06::exo-reventeVoiture"`. Soumettez ensuite l'ensemble des `commit` au dépôt.